

学习笔记

第二章：数据组织、写入与表设计

沿着 Table - Partition - Tablet - Rowset - Segment - Page 进入存储底层，再把副本、索引、Compaction、数据模型和事务可见性组合成一套可落地的表设计方法。

根据本章逐节讨论重新整理

基准版本：Apache Doris 3.0.8

整理日期：2026-08

目录

阅读方式	4
2.1 从 Table 到 Page：完整存储层次	4
一行订单的落盘路径	5
逻辑对象与物理对象不要混淆	5
2.2 Partition：范围管理与第一层裁剪	6
为什么分析表常按时间分区	6
分区裁剪如何发生	6
常见误区	6
选择粒度的设计思想	6
2.3 Bucket 与 Tablet：把一个分区分散到集群	7
Hash 分桶的两个关键效果	7
分桶键如何选择	7
桶数为什么不能拍脑袋	7
分区剪枝和桶剪枝的顺序	7
2.4 Replica：多数派写入、故障容忍与修复	8
三副本写入的直观过程	8
为什么不要全部副本同时成功	8
读取是不是随机找一个副本	8
副本数的代价	8
修复为什么也可能拖慢业务	8
2.5 Rowset、Segment 与 Page：不可变写入的物理组织	8
从微批数据到磁盘文件	8
为什么采用不可变 Segment	9
Segment 内部为什么还要分 Page	9
版本链示例	9

2.6 Compaction：用后台写放大控制前台读放大	9
它在交换什么	9
Cumulative 与 Base 的直观分工	10
为什么微批不能小到逐行	10
积压时的典型症状	10
排查顺序	10
2.7 索引与 Data Skipping：快来自少读，不是读完再算快	10
一条查询的逐层削减	10
常用索引对比	11
Prefix Index 为什么依赖列顺序	11
索引不是越多越好	11
2.8 数据模型：重复、聚合与主键更新的语义选择	11
Duplicate Key：重复是业务事实	11
Aggregate Key：写入的是可合并增量	11
Unique Key MOW：写时处理旧版本	12
为什么 CDC 还需要 Sequence Column	12
3.0.8 需要特别记住的限制	12
选型判断	12
2.9 事务、版本与查询快照	13
五种状态	13
Transaction ID、Label 与 Version	13
Label 如何帮助重试	13
Publish Timeout 不等于写入失败	13
查询快照	14
MOW 与快照怎样共存	14
2.10 综合实战：订单实时数仓的三表设计	14
需求到设计的映射	15

ODS：完整事件历史	15
DWD：订单当前状态	16
ADS：商家日报	16
Aggregate 输入必须是增量	16
一条 CDC 事件如何流过三张表	17
是否把三张表放进一个事务	17
微批与 Group Commit	17
建表决策顺序	17
故障诊断速查	17
本章总结	18
十五个必须记住的结论	18
综合复习题	18
术语速查	19
参考资料	19

阅读方式

本章不把建表语句当成参数清单，而是追踪一行数据如何被组织、复制、写入、合并、索引并最终进入查询快照。理解这条链路后，很多性能与正确性问题都可以从底层推导。

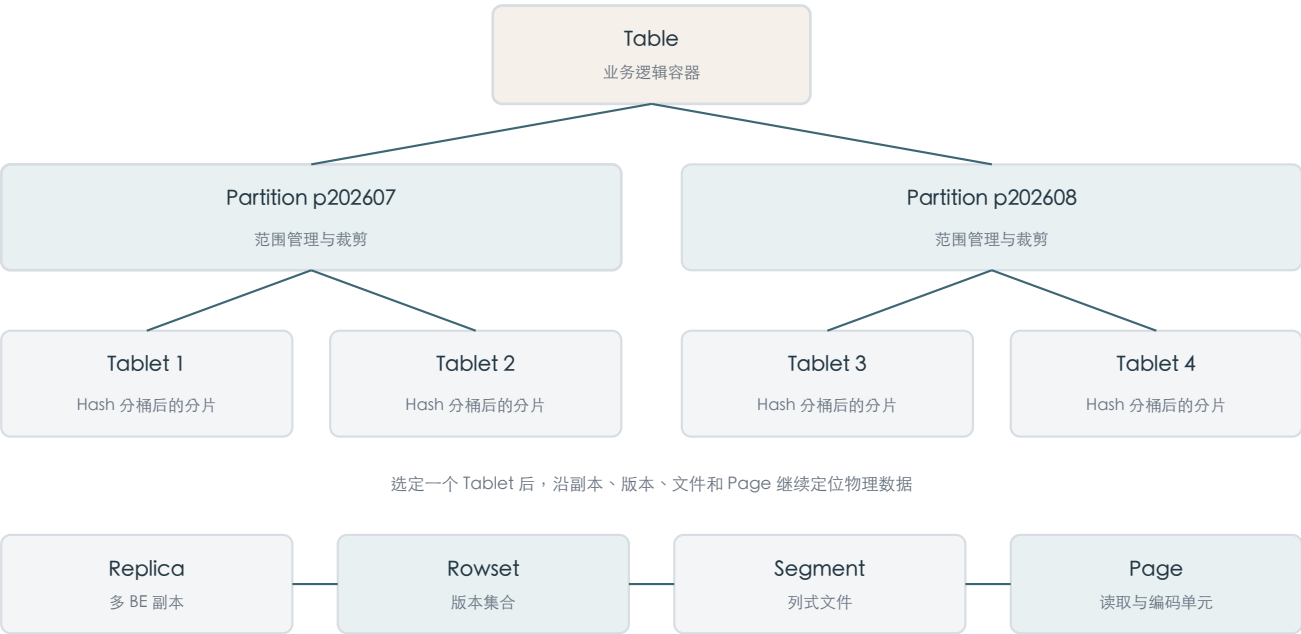
- 逻辑层：Table、Partition、Key 模型描述业务语义和数据管理边界。
- 分布层：Bucket、Tablet、Replica 决定数据落在哪些节点以及如何容错。
- 物理层：Rowset、Segment、Page 决定写入、扫描、压缩和索引的实际成本。
- 时间层：Transaction、Version、Snapshot 决定数据什么时候提交、可见以及被哪次查询读取。

贯穿全章的一句话：表设计决定数据放在哪里；事务和版本决定数据何时可见；索引决定查询少读多少；Compaction 决定这种性能能维持多久。

2.1 从 Table 到 Page：完整存储层次

Doris 的数据并不是以一个巨大的表文件存在。它被逐层划分：先形成管理边界，再形成分布式分片，最后落为不可变的列式文件与 Page。

从逻辑表到磁盘 Page



层次	核心职责	最常回答的问题
Table	承载 Schema、数据模型、分区和索引定义	这张表表示什么业务事实？
Partition	按时间或业务范围划分管理单元	哪些历史范围可以裁剪或删除？
Bucket / Tablet	在分区内按 Hash 水平切分	一行数据应落到哪个分片？
Replica	同一 Tablet 在不同 BE 上的副本	节点故障后如何继续写入和读取？
Rowset	一次写入或 Compaction 形成的版本集合	本次事务生成了哪些不可变数据？
Segment	Rowset 内实际的列式存储文件	扫描、索引和编码在哪里发生？
Page	Segment 内更细的读取、压缩和统计单元	能否再跳过一小段数据？

一行订单的落盘路径

```
orders
-> partition p202608
-> hash(order_id) -> tablet 17
-> replica on BE-1 / BE-2 / BE-3
-> pending rowset of transaction 5008
-> segment files
-> encoded column pages
```

这条路径解释了为什么一个 SQL 可以逐层缩小范围：分区谓词先排除大范围历史数据，分桶谓词再减少 Tablet，Segment 与 Page 的统计和索引继续跳过物理数据。

逻辑对象与物理对象不要混淆

- Partition 和 Tablet 是查询计划、数据管理与分布式调度能看到的逻辑分片。
- Replica 是 Tablet 的物理冗余实例，不是另一份独立业务数据。
- Rowset 和 Segment 更接近磁盘上的版本与文件；Page 则是文件内部的读取单元。

Partition 解决范围管理，Tablet 解决横向分布，Replica 解决容错，Rowset 解决版本化写入，Segment 和 Page 解决高效列式读取。

2.2 Partition：范围管理与第一层裁剪

Partition 是表内最大的显式数据管理边界。它通常按日期、月份或有限业务范围划分，使查询、数据生命周期和运维动作都能以一块数据为单位进行。

为什么分析表常按时间分区

- 查询往往自带时间范围，可以直接排除无关分区。
- 过期数据可以通过删除分区快速下线，不必逐行 DELETE。
- 冷热策略、备份、恢复和数据修正可以围绕时间范围组织。
- 新数据集中进入少量活跃分区，历史分区逐渐稳定。

```

PARTITION BY RANGE(event_date) (
  PARTITION p202608
  VALUES [('2026-08-01'), ('2026-09-01'))
)

```

分区裁剪如何发生

FE 分析 WHERE 条件后，把能确定无关的分区从扫描计划中移除。对于 24 个月数据，如果查询只命中一天，合理的分区设计可让后续 Tablet、Segment 和 Page 根本不接触其他月份。

```

SELECT merchant_id, SUM(amount)
FROM orders
WHERE event_date >= '2026-08-15'
  AND event_date < '2026-08-16'
GROUP BY merchant_id;

```

常见误区

误区	后果	修正思路
分区越细越好	分区元数据和调度对象膨胀	让分区规模与保留、回补和查询范围匹配
用高基数字段分区	产生大量碎片化分区	高基数均匀字段更适合 Hash 分桶
查询不带分区条件	所有历史分区仍可能参与	在 API 和语义层强制时间范围
分区列会被业务更新	同一逻辑主键可能落入不同分区	选择创建日期等稳定字段
对分区列做复杂表达式	可能影响谓词推导和裁剪	优先使用直接范围条件

选择粒度的设计思想

日分区、月分区并没有固定答案。应同时考虑单日数据量、常见查询跨度、回补范围、保留周期以及分区数量。高吞吐大表常用日分区；数据较少、保留很久的表可用月分区。

分区的第一目标是管理和裁剪，不是把数据平均分到机器。机器之间的均衡由 Tablet、Replica 和调度系统继续完成。

2.3 Bucket 与 Tablet：把一个分区分散到集群

一个分区仍可能包含数亿行，不能作为一个整体交给一台机器。Doris 使用分桶表达分布规则，并把每个桶实现为 Tablet。

先按日期找分区，再按 Hash 找 Tablet



Hash 分桶的两个关键效果

1. 稳定路由：相同分桶键经过同一 Hash 规则后进入同一桶。
2. 并行执行：多个 Tablet 可以分布在多个 BE 上，同时扫描和聚合。

DISTRIBUTED BY HASH(order_id) BUCKETS 32

这里的 32 表示每个分区拥有 32 个 Tablet，而不是全表一共 32 个。如果表有 100 个分区，基础 Tablet 数量就是 3200，再乘副本数才是 Replica 数量。

分桶键如何选择

目标	更合适的特征	需要避免
数据均匀	高基数、分布稳定	性别、状态等低基数字段
点查剪枝	常见等值过滤字段	查询从不携带的字段
局部聚合或 Join	业务相关且两表规则可对齐	随业务频繁变化的字段
降低热点	能打散大客户或大商家	极端头部 Key 单独成热点

桶数为什么不能拍脑袋

- 桶太少：单 Tablet 过大、并行度不足、迁移和修复变慢。
- 桶太多：大量小 Tablet 增加元数据、调度、文件与 Compaction 开销。
- 应根据每个分区的数据量、BE 数量、并发和未来增长选择，并通过真实 Tablet 大小持续校准。

分区剪枝和桶剪枝的顺序

```
WHERE order_date = '2026-08-15'
      AND order_id = 1001
```

日期条件先选中目标分区；order_id 等值条件再根据 Hash 选择目标 Tablet。之后才轮到排序、索引、Segment 和 Page 级别的数据跳过。

Partition 是范围切割；Bucket 是 Hash 规则；Tablet 是 Hash 规则产生的实际数据分片。

2.4 Replica：多数派写入、故障容忍与修复

Replica 是同一 Tablet 的多个物理副本。默认三副本并不意味着业务数据出现三条，而是同一份逻辑分片在不同 BE 上保存三份，以容忍节点和磁盘故障。

三副本写入的直观过程

tablet 17

```
replica A on BE-1 -> success
replica B on BE-2 -> success
replica C on BE-3 -> timeout
```

majority = 2 of 3 -> transaction may commit

多数派设计让一个慢副本不会无限拖住写入。事务提交后，异常副本被标记并由后台克隆或修复机制追上缺失版本。

为什么不要全部副本同时成功

- 要求全部成功会让最慢节点决定集群写入可用性。
- 完全不要求多数派又可能在进一步故障时失去可靠数据。
- 多数派是在一致性、延迟和可用性之间做出的工程折中。

读取是不是随机找一个副本

调度器会根据副本健康状态、版本完整性、节点负载和位置等因素选择可读取副本。一个存活但版本落后的副本不能被当作完整数据源。

副本数的代价

副本数	能力	代价与风险
1	空间与写入成本最低	节点或磁盘故障即可造成不可用或丢失
2	拥有冗余副本	多数派容忍度有限，维护窗口更谨慎
3	常见生产折中，可容忍单点异常	约 3 倍主数据空间与写放大

修复为什么也可能拖慢业务

副本修复需要读取健康副本并在目标 BE 写入数据，会占用网络、磁盘和后台线程。故障恢复期间既要保证业务查询，也要尽快恢复冗余，因此修复速度和前台负载需要平衡。

Replica 解决的是同一 Tablet 的高可用；事务是否提交取决于足够副本成功，副本是否可读还取决于其版本是否完整。

2.5 Rowset、Segment 与 Page：不可变写入的物理组织

Doris 不会在原文件中逐行原地修改。一次导入通常生成新的 Rowset，Rowset 内包含一个或多个 Segment，Segment 再由列数据 Page、索引和元数据组成。

从微批数据到磁盘文件


```

input batch
  -> parse and route
  -> MemTable
  -> sort / aggregate / deduplicate by model
  -> flush Segment
  -> assemble pending Rowset
  -> publish version
  -> visible Rowset

```

为什么采用不可变 Segment

- 顺序写入和批量压缩更适合分析型存储。
- 正在执行的查询可以继续持有旧 Rowset，避免文件原地变化。
- 新数据通过新版本追加，事务发布和故障恢复更清晰。
- 代价是 Rowset 持续累积，需要 Compaction 定期整理。

Segment 内部为什么还要分 Page

如果整列是一个不可分割的大块，查询只需要少量范围时仍要解压很多数据。Page 让编码、压缩、缓存和统计信息拥有更细粒度，同时避免每行一个块造成元数据爆炸。

对象	典型内容	对查询的意义
Rowset	事务编号、版本范围、多个 Segment	构成 Tablet 的可见版本链
Segment	列数据、短 Key 索引、倒排等索引与 Footer	主要文件扫描与索引定位单元
Page	一段列值、编码压缩数据、局部统计	按需读取、解码并继续跳过

版本链示例

before compaction: [0-8] + [9-9] + [10-10] + [11-11]
 after compaction : [0-11]

Compaction 改写了物理文件，却保持逻辑版本覆盖不变。对于查询快照而言，这两条读取路径必须得到相同结果。

Rowset 是版本集合，Segment 是实际列式文件，Page 是文件内部的读取与编码单元。一次提交可能在多个 Tablet 上分别产生 Rowset。

2.6 Compaction：用后台写放大控制前台读放大

持续导入会产生越来越多小 Rowset。查询若长期同时打开大量 Rowset 和 Segment，文件句柄、索引读取、合并与 Delete Bitmap 检查都会增加。Compaction 将多个不可变 Rowset 合并为更少、更大的 Rowset。

它在交换什么

```

many small rowsets
  -> read old segments
  -> merge / rewrite
  -> create larger rowset
  -> switch metadata
  -> reclaim old files later

```

本质上，Compaction 用后台 CPU、磁盘读取和写入换取更少的查询文件与更短的版本链。这是典型的写放大换读放大。

Cumulative 与 Base 的直观分工

类型	主要对象	设计目标
Cumulative Compaction	较新的增量 Rowset	快速合并近期小版本，控制版本增长
Base Compaction	Base 与更大范围历史 Rowset	形成更大的长期基础版本，整理历史数据

为什么微批不能小到逐行

每条数据一个事务会制造大量版本和小文件。即使单次写入看起来很快，Compaction 迟早要支付合并成本。合理的秒级微批或 Group Commit 能在实时性与后台成本之间取得平衡。

积压时的典型症状

- 查询逐渐变慢，扫描的 Rowset 和 Segment 数持续上升。
- 导入报版本过多或等待，写入延迟出现明显抖动。
- 磁盘使用量下降缓慢，因为旧文件尚未完成合并与回收。
- 后台 Compaction 队列与得分持续偏高。

排查顺序

1. 先确认写入是否过碎：批次大小、并发、每分钟事务数和版本增长速度。
2. 检查 Rowset、Segment 数量以及哪些 Tablet 最严重，避免只看全局平均。
3. 检查 BE 磁盘空间、I/O、CPU 和后台线程，确认是否有倾斜或资源饥饿。
4. 再评估表模型、索引数量、分桶规模以及 Compaction 策略。

Compaction 不是修复错误模型的魔法。真正的起点通常是让每次写入形成足够合理的批次和 Tablet 分布。

2.7 索引与 Data Skipping：快来自少读，不是读完再算快

Doris 的索引大多不是 OLTP 中定位一行的 B+Tree，而是分布在不同层次的数据跳过机制。它们共同回答：哪些分区、Tablet、Segment、Page 或行号范围根本不需要读取。

一条查询的逐层削减

```
all table data
-> partition pruning
-> tablet pruning
-> prefix / zone-map skipping
-> bloom / inverted / ngram index
-> read selected pages and columns
-> vectorized filter and aggregate
```

常用索引对比

机制	适合条件	核心思想	局限
Prefix Index	符合 Key 前缀的等值与范围过滤	利用排序后的短 Key 快速定位范围	跳过中间列后，后续列通常难继续使用前缀
Zone Map	有局部有序性或数据聚集的范围过滤	用 Page/块的 min、max 排除不可能命中区域	数据完全随机时效果变弱
Bloom Filter	高基数等值查询	快速判断某数据块一定不存在目标值	可能误判存在，且不适合范围查询
Inverted Index	字符串、文本、数值等灵活条件	值或词项映射到行号集合	占空间并增加写入和 Compaction 成本
NGram Bloom	LIKE 包含匹配	把字符串切成 NGram 后进行概率过滤	短模式或选择性差时收益有限

Prefix Index 为什么依赖列顺序

```
DUPLICATE KEY(event_date, merchant_id, event_time, order_id)
```

```
good : event_date = ? AND merchant_id = ?
weak : order_id = ?
```

排序 Key 让相邻记录在磁盘上拥有业务相关的局部性。查询从最左列开始约束时，能逐步缩小有序范围；只过滤靠后的 order_id 时，前缀无法直接定位，因此可用分桶和倒排索引补充。

索引不是越多越好

- 每个索引都需要写入构建、磁盘空间、内存元数据和 Compaction 重写。
- 低选择性条件即使命中索引，仍可能返回大量行号，收益有限。
- 应通过 EXPLAIN、Query Profile 和扫描字节验证，而不是仅凭字段重要性建索引。

正确的优化顺序通常是：先分区和分桶，再设计 Key 排序，最后为剩余高价值查询补充索引。

2.8 数据模型：重复、聚合与主键更新的语义选择

Doris 的 Key 模型首先定义同 Key 数据写入后应该如何解释。模型选错是正确性问题，不只是性能问题。

模型	同 Key 写入结果	典型用途
Duplicate Key	全部保留	日志、事件流水、明细事实、原始 CDC
Aggregate Key	按列定义 SUM、MAX、MIN 等聚合	可加指标、预聚合报表
Unique Key MOW	主键只保留最新有效版本	用户画像、订单当前状态、CDC 同步

Duplicate Key：重复是业务事实

订单从 CREATED 变为 PAID 再变为 SHIPPED，这三条事件都应保留。Duplicate 表不会替你去重；Key 主要承担排序与索引前缀作用。

Aggregate Key：写入的是可合并增量

```
AGGREGATE KEY(stat_date, merchant_id)
paid_order_count BIGINT SUM
paid_gmv DECIMAL(20, 2) SUM
last_event_time DATETIMEV2(3) MAX
```

Aggregate 表必须明确输入是增量还是快照。把同一订单的完整金额快照重复送入 SUM，会得到业务错误的翻倍结果。

Unique Key MOW：写时处理旧版本

Merge-on-Write 在写入和发布阶段识别同主键旧记录，并通过 Delete Bitmap 标记其在新版本中失效。查询只需读取新快照中的有效行，避免每次查询再做全局去重。

为什么 CDC 还需要 Sequence Column

```
arrive first : 12:05 SHIPPED
arrive later : 12:02 PAID
```

sequence column decides SHIPPED is newer

没有业务序列时，最后到达的数据可能错误覆盖较新状态。Sequence Column 应优先使用源数据库 LSN、binlog position 或稳定递增版本，而不是仅依赖到达顺序。

3.0.8 需要特别记住的限制

- Unique Key 在 3.0 默认采用 Merge-on-Write，建表时仍建议显式写出属性，便于审阅。
- 3.0.x 的一次部分列更新批次中，各行应更新相同的列集合；更灵活的行级不同列集合属于后续版本能力。
- 从 3.0.6 起 Unique 表禁止使用 time_series Compaction，3.0.8 应使用 size_based。

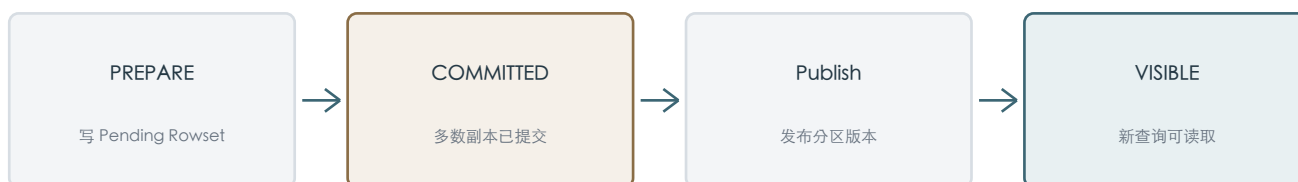
选型判断

需要历史明细选 Duplicate；同 Key 的数值天然可交换、可结合地合并时选 Aggregate；需要按业务主键维护当前状态时选 Unique MOW。

2.9 事务、版本与查询快照

在 Doris 中，文件已经写入、事务已经提交、查询已经可见是三个不同阶段。理解这一区别，是排查“导入成功却查不到”问题的关键。

一次导入从写入到查询可见



COMMITTED 表示不能再正常回滚；VISIBLE 才表示查询可见

五种状态

状态	含义	查询能否读取
PREPARE	事务已创建，数据可能正在写 Pending Rowset	不能
PRECOMMITTED	Stream Load 2PC 等待外部 commit 或 abort	不能
COMMITTED	多数副本写入成功，事务不能正常回滚，但尚未完成发布	不一定
VISIBLE	版本发布完成，Rowset 进入可见版本链	新查询可以
ABORTED	事务失败或被取消，临时数据后续清理	不能

Transaction ID、Label 与 Version

概念	回答的问题	是否全局可比
Transaction ID	这是 Doris 内部哪一次事务？	事务编号用于管理，不代表数据快照
Label	客户端如何识别批次并保证幂等？	在数据库导入范围内承担唯一标识
Version	某分区或 Tablet 的逻辑数据推进到哪里？	不同分区的相同数字不能比较
Rowset Version	这个 Rowset 覆盖哪个版本区间？	只在所属版本链中解释

Label 如何帮助重试

orders_kafka_p3_1000_1999

same source partition + same offset range -> same label

网络超时后应先用原 Label 查询事务状态：VISIBLE 表示完成；COMMITTED 表示继续等待发布，不能重发；ABORTED 表示修复原因后发起新尝试；未知状态下也优先保留同一确定性 Label，避免双写。

Publish Timeout 不等于写入失败

Publish Timeout 往往表示提交已成功，只是没有在客户端超时时间内变为 VISIBLE。若此时换新 Label 重发，Duplicate 表可能产生重复数据，Aggregate 表可能重复累加。

```
SHOW TRANSACTION FROM order_db
WHERE LABEL = 'orders_kafka_p3_1000_1999';
```

```
SHOW LAST INSERT;
```

查询快照

Q1 starts -> captures visible version V10
load publishes V11 while Q1 is running
Q1 continues reading V10
Q2 starts later -> captures V11

Doris 使用 READ COMMITTED 语义，每条语句开始时捕获快照。查询过程中新的提交不会让同一条语句前后读取不同版本。显式多语句事务也不应被当成 MySQL 的长时间可重复读快照。

MOW 与快照怎样共存

V10 中订单状态为 CREATED；V11 写入 PAID 并让 Delete Bitmap 从 V11 开始屏蔽旧行。V10 查询仍看到 CREATED，V11 查询看到 PAID。删除标记必须与版本绑定，才能同时服务正在运行的旧查询和新查询。

COMMITTED 解决持久性，VISIBLE 解决可读性，Label 解决客户端幂等，Version 与 Snapshot 解决一致读取。

2.10 综合实战：订单实时数仓的三表设计

案例要求同时支持订单历史追踪、当前状态查询和商家日报。三种需求的数据语义不同，因此分别使用 Duplicate、Unique MOW 和 Aggregate。

订单 CDC 的三种数据语义



需求到设计的映射

需求	设计选择	底层原因
保留每次 CDC 事件	ODS Duplicate	重复事件本身是需要审计的事实
每笔订单一行最新状态	DWD Unique MOW + Sequence	写入阶段处理主键旧版本和乱序
商家日报低延迟查询	ADS Aggregate	提前把明细合并为日期、商家粒度
按时间回补和过期	Range 时间分区	范围裁剪与生命周期管理
订单点查与数据均衡	HASH(order_id)	高基数、稳定路由并支持桶剪枝
抗单节点故障	三副本	多数派提交与后台修复

ODS：完整事件历史

```
CREATE TABLE ods_order_event (  
  event_date DATE NOT NULL,  
  merchant_id BIGINT NOT NULL,  
  event_time DATETIMEV2(3) NOT NULL,  
  order_id BIGINT NOT NULL,  
  op_type VARCHAR(16),  
  status VARCHAR(32),  
  amount DECIMAL(18, 2),  
  source_lsn BIGINT,  
  raw_json STRING,  
  INDEX idx_order_id (order_id) USING INVERTED  
)  
DUPLICATE KEY(event_date, merchant_id, event_time, order_id)  
PARTITION BY RANGE(event_date) (  
  PARTITION p202608  
    VALUES [('2026-08-01'), ('2026-09-01'))  
)  
DISTRIBUTED BY HASH(order_id) BUCKETS 32  
PROPERTIES (  
  "replication_num" = "3",  
  "compaction_policy" = "time_series"  
);
```

时间与商家位于排序前缀，适合按日期和商家追踪事件；order_id 负责分桶，并以倒排索引补充不带前缀的订单查询。桶数 32 只是示例，生产中必须按分区规模和 BE 数量校准。

DWD：订单当前状态

```
CREATE TABLE dwd_order_current (
    create_date DATE NOT NULL,
    merchant_id BIGINT NOT NULL,
    order_id BIGINT NOT NULL,
    user_id BIGINT,
    status VARCHAR(32),
    amount DECIMAL(18, 2),
    payment_no VARCHAR(64),
    update_time DATETIMEV2(3) NOT NULL,
    INDEX idx_order_id (order_id) USING INVERTED
)
UNIQUE KEY(create_date, merchant_id, order_id)
PARTITION BY RANGE(create_date) (
    PARTITION p202608
    VALUES [('2026-08-01'), ('2026-09-01'))
)
DISTRIBUTED BY HASH(order_id) BUCKETS 32
PROPERTIES (
    "enable_unique_key_merge_on_write" = "true",
    "function_column.sequence_col" = "update_time",
    "bloom_filter_columns" = "payment_no",
    "replication_num" = "3",
    "compaction_policy" = "size_based"
);
```

create_date 和 merchant_id 必须在订单生命周期内稳定。如果更新事件给出不同 Key，Doris 会把它当作另一行。更严格的 CDC 可把源 LSN 作为 Sequence Column，避免相同毫秒时间或时钟问题。

ADS：商家日报

```
CREATE TABLE ads_merchant_day (
    stat_date DATE NOT NULL,
    merchant_id BIGINT NOT NULL,
    paid_order_count BIGINT SUM DEFAULT "0",
    paid_gmv DECIMAL(20, 2) SUM DEFAULT "0",
    refund_count BIGINT SUM DEFAULT "0",
    last_event_time DATETIMEV2(3) MAX
)
AGGREGATE KEY(stat_date, merchant_id)
PARTITION BY RANGE(stat_date) (
    PARTITION p202608
    VALUES [('2026-08-01'), ('2026-09-01'))
)
DISTRIBUTED BY HASH(merchant_id) BUCKETS 16
PROPERTIES (
    "replication_num" = "3",
    "compaction_policy" = "size_based"
);
```

Aggregate 输入必须是增量

```
first payment      : order_count +1, gmv +100
payment cancelled : order_count -1, gmv -100
amount 100 -> 120 : order_count 0, gmv +20
```

如果上游把 100 元完整快照重复发送两次，SUM 会正确地计算为 200，但业务语义是错误的。应生成补偿增量、由去重后的任务写入，或重算整个目标分区。

一条 CDC 事件如何流过三张表

事件	ODS	DWD	ADS
订单 1001 从 CREATED 变为 PAID，金额 100	追加 PAID 事件，CREATED 仍保留	PAID 新版本生效，旧状态被 Delete Bitmap 屏蔽	支付订单 +1，GMV +100
更旧的 CREATED 事件迟到	继续保留，便于审计	Sequence 阻止状态倒退	不应再次产生支付增量

是否把三张表放进一个事务

只有确实要求三表同时可见时才考虑显式多表事务，而且应由上游预先准备三份写入数据。不要依赖事务内先写 ODS、再从刚写的 ODS 查询生成 DWD，因为 Doris 的显式事务不能简单当作带 read-your-writes 的 MySQL 长事务。

更常见的实时数仓做法是为 ODS、DWD、ADS 分别定义新鲜度 SLA，通过 Kafka Offset、Flink Checkpoint、Stream Load 2PC、确定性 Label 和 Sequence Column 实现可恢复的最终一致。

微批与 Group Commit

逐行事务会快速制造版本和小 Rowset。实践中常按数百到数万行或 1 到 5 秒形成微批；高并发小写可评估 Group Commit。但显式 Label、2PC 和部分列更新等场景可能走非 Group Commit 路径，Exactly Once 与合并小事务需要按链路设计权衡。

建表决策顺序

1. 先定义一行代表什么业务事实，以及重复数据应保留、聚合还是覆盖。
2. 选择 Duplicate、Aggregate 或 Unique MOW，并确定稳定 Key。
3. 根据保留、回补和常见时间条件选择分区。
4. 根据均匀性、点查和 Join 选择分桶键与桶数。
5. 根据高频过滤条件设计 Key 顺序和排序局部性。
6. 只为前缀与 Zone Map 无法覆盖的高价值查询增加索引。
7. 设计副本、Label、Sequence、批次和失败恢复契约。
8. 用 EXPLAIN、Query Profile、Rowset 和 Compaction 指标验证真实效果。

故障诊断速查

现象	优先检查	常见根因
导入成功但查不到	SHOW TRANSACTION	事务仍为 COMMITTED，版本尚未发布
当前状态倒退	Sequence Column 与源序列	CDC 乱序或序列值不可靠
Unique 出现两行	完整 Unique Key	分区列或其他 Key 字段发生变化
GMV 翻倍	ADS 输入语义	把完整快照当成 SUM 增量
订单点查慢	分区、桶、索引命中	缺时间范围、未桶剪枝或索引无效
查询越来越慢	Rowset、Segment、Compaction	微批过小、后台合并积压
某个 BE 特别忙	Tablet 与 Key 分布	分桶倾斜或热点 Key
磁盘回收缓慢	Compaction 与文件回收	逻辑失效文件尚未完成物理回收

模型和 Key 保证语义，Partition 与 Bucket 保证范围和分布，Replica 保证可用性，Rowset 与 Version 保证不可变写入和一致读取，索引与 Compaction 保证长期性能。

本章总结

十五个必须记住的结论

1. Table 到 Page 是从业务逻辑到物理读取单元的逐层组织。
2. Partition 解决范围管理与裁剪，不负责机器间的平均分布。
3. Bucket 是 Hash 规则，Tablet 是规则产生的实际分片。
4. Replica 是同一 Tablet 的冗余实例，不是独立业务记录。
5. 三副本多数派让一个异常节点不必阻塞所有写入。
6. 一次导入形成新 Rowset；Rowset 内包含一个或多个 Segment。
7. Segment 不原地修改，使事务版本、快照和并发查询更容易保证。
8. Compaction 用后台写放大换取更低的前台读放大。
9. 索引体系的共同目标是少读数据，而不是读取全部后再快速计算。
10. Duplicate、Aggregate、Unique 首先是三种业务语义。
11. Unique MOW 把主键去重成本放到写入侧，Sequence 处理乱序。
12. 文件写完、事务 COMMITTED 和数据 VISIBLE 是不同阶段。
13. Label 提供导入批次幂等，Publish Timeout 后不能盲目换 Label 重发。
14. 每条查询在开始时捕获快照，Compaction 和新提交不能改变其逻辑结果。
15. 一套好表设计从数据语义开始，最后才是索引和参数。

综合复习题

1. Partition 和 Bucket 分别解决什么问题？为什么不能互相替代？
2. 为什么一个表的基础 Tablet 数等于各分区桶数之和，而 Replica 数还要乘副本数？
3. 三副本中一个副本写入超时，事务为什么仍可能提交？之后如何恢复？
4. Rowset、Segment、Page 分别承担什么职责？
5. Compaction 为什么既能改善查询，也可能与导入争抢资源？
6. Prefix、Bloom Filter、Inverted Index 分别适合什么查询？
7. Duplicate、Aggregate、Unique MOW 在同 Key 写入时有何差异？
8. 没有 Sequence Column 时，乱序 CDC 会造成什么问题？
9. COMMITTED 和 VISIBLE 的区别是什么？Publish Timeout 后应该怎么做？
10. 为什么查询 Q1 在运行中遇到 V11 发布，仍继续读取 V10？
11. 为什么 Aggregate 表不能直接重复接收当前状态快照？
12. 面对一个查询慢问题，你会按哪些裁剪与存储层次逐步排查？

术语速查

术语	一句话解释
Partition	按范围划分的数据管理和裁剪单元
Bucket	分区内的 Hash 分布规则
Tablet	Bucket 对应的实际水平数据分片
Replica	同一 Tablet 在某个 BE 上的物理副本
Rowset	一次写入或 Compaction 形成的版本集合
Segment	Rowset 内承载列数据和索引的不可变文件
Page	Segment 内的读取、编码和压缩单元
Delete Bitmap	Unique MOW 中按版本标记旧行失效的结构
Compaction	把多个 Rowset 重写合并为更少 Rowset 的后台过程
Label	客户端为导入批次提供的幂等标识
Version	分区或 Tablet 数据状态的逻辑进度
Snapshot	查询开始时捕获的一致可见版本

参考资料

以下均为 Apache Doris 官方文档或 Release Notes。本笔记以 3.0.8 为基准，部分持续维护的原理页面位于 3.x 路径。

- [1] Basic Concepts: Partition and Bucket
<https://doris.apache.org/docs/3.x/table-design/data-partitioning/basic-concepts/>
- [2] Table Bucketing
<https://doris.apache.org/docs/3.x/table-design/data-partitioning/data-bucketing/>
- [3] Load High Availability
<https://doris.apache.org/docs/3.x/data-operate/import/load-high-availability/>
- [4] Rowsets System Table
https://doris.apache.org/docs/3.0/admin-manual/system-tables/information_schema/rowsets
- [5] Compaction Troubleshooting
<https://doris.apache.org/docs/3.x/admin-manual/trouble-shooting/compaction/>
- [6] Prefix Index
<https://doris.apache.org/docs/3.x/table-design/index/prefix-index/>
- [7] Bloom Filter Index
<https://doris.apache.org/docs/3.x/table-design/index/bloomfilter/>
- [8] Inverted Index
<https://doris.apache.org/docs/table-design/index/inverted-index>
- [9] NGram Bloom Filter Index
<https://doris.apache.org/docs/3.0/table-design/index/ngram-bloomfilter-index>
- [10] Data Model Overview
<https://doris.apache.org/docs/3.x/table-design/data-model/overview/>
- [11] Unique Key Table
<https://doris.apache.org/docs/3.0/table-design/data-model/unique/>
- [12] Unique Update Concurrent Control
<https://doris.apache.org/docs/3.0/data-operate/update/unique-update-concurrent-control/>
- [13] Partial Column Update
<https://doris.apache.org/docs/3.x/data-operate/update/partial-column-update/>
- [14] Transaction
<https://doris.apache.org/docs/3.0/data-operate/transaction/>

[15] SHOW TRANSACTION

<https://doris.apache.org/docs/3.0/sql-manual/sql-statements/transaction/SHOW-TRANSACTION>

[16] Stream Load Principle

<https://doris.apache.org/blog/principle-of-Doris-Stream-Load/>

[17] Group Commit

<https://doris.apache.org/docs/3.x/data-operate/import/group-commit-manual/>

[18] Release 3.0.6

<https://doris.apache.org/releases/v3.0/release-3.0.6/>

[19] Release 3.0.8

<https://doris.apache.org/releases/v3.0/release-3.0.8/>